

Object Oriented Programming with Java

Unit IV: Java Collections Framework

AKTU BCS-403

Contents

1	Introduction to Collections in Java	4
1.1	Key Features of Collections	4
2	What is a Framework? What is the Collection Framework?	4
2.1	Framework in Java	4
2.2	Collection Framework	4
2.3	Advantages of the Java Collection Framework	4
3	Hierarchy of the Collection Framework	5
4	Iterator Interface	5
4.1	Purpose of the Iterator Interface	5
5	Collection Interface	5
6	Difference Between Collection and Collections	6
7	List Interface	6
7.1	Key Characteristics of List Interface	6
8	ArrayList	6
8.1	How ArrayList Works	7
8.2	Example: ArrayList Traversal with Iterator	7
9	LinkedList	7
9.1	How LinkedList Works	8
9.2	Example: LinkedList Traversal with Iterator	8
10	Vector	8
10.1	Example: Vector	9
11	Stack	9
11.1	Example: Stack	10
12	Queue Interface	10

12.1 Instantiation Examples	11
13 PriorityQueue	11
13.1 Example: PriorityQueue	11
14 Deque Interface	12
14.1 Instantiation Example	12
15 ArrayDeque	12
15.1 Example: ArrayDeque	13
16 Set Interface	13
16.1 Instantiation Examples	14
17 HashSet	14
17.1 Declaration of HashSet	14
17.2 Example: HashSet	14
18 LinkedHashSet	15
19 SortedSet	15
20 TreeSet	15
21 Map Interface	16
22 HashMap	16
23 LinkedHashMap	16
24 Hashtable	17
25 Sorting in Java	17
26 Comparable Interface	17
27 Sorting with Collections.sort()	18
27.1 Method Signature	18
28 Comparator Interface	18
28.1 Syntax	18
29 Sorting List Elements Using Comparator	19
29.1 How the sort() Method Works	19
30 Properties Class	19
30.1 Advantages of a Properties File	19
30.2 Important Note	19
30.3 Declaration	19
30.4 Constructors of Properties	19

31 Conclusion**20**

1. Introduction to Collections in Java

A collection in Java represents a single unit of objects, that is, a group of objects. The Java Collections Framework provides an architecture to store and manipulate groups of objects. It supports common operations such as searching, sorting, insertion, deletion, traversal, and data manipulation.

In Java, the term *Collections* commonly refers to the framework available in the Java API for managing and manipulating groups of objects. These objects are usually called *elements* or *items*. The framework provides a unified way to store, retrieve, update, and iterate over groups of data.

1.1 Key Features of Collections

1. **Unified Architecture:** The framework provides a standard structure for representing and manipulating collections.
2. **Generics Support:** Most collection classes and interfaces support generics, allowing the programmer to specify the type of elements stored in the collection.
3. **Dynamic Resizing:** Many collection classes resize automatically when elements are inserted or removed.
4. **Algorithms Support:** The framework includes utility methods and algorithms for sorting, searching, and efficient data manipulation.
5. **Iterators:** Iterators provide a standard way to traverse elements in a collection.

2. What is a Framework? What is the Collection Framework?

2.1 Framework in Java

A framework in Java provides a ready-made architecture. It represents a set of classes and interfaces designed to solve a category of programming problems in a standard way. It is optional, but highly useful, because it reduces repeated development effort.

2.2 Collection Framework

The Java Collection Framework is a set of classes and interfaces that provide reusable implementations of common data structures such as lists, sets, maps, and queues. It gives a unified architecture for storing and manipulating groups of objects.

2.3 Advantages of the Java Collection Framework

1. Reusable implementations
2. Consistency
3. Efficiency
4. Type safety

5. Scalability

3. Hierarchy of the Collection Framework

The package `java.util` contains the classes and interfaces required for the Collection Framework.

The hierarchy begins with the `Iterable` interface. This interface provides the facility to obtain an iterator and traverse collection elements. The `Collection` interface extends `Iterable` and acts as the main root of the collection hierarchy.

The main branches of the collection hierarchy are:

- **List**
- **Queue**
- **Set**

Common implementations shown in the unit material include:

- For **List**: `ArrayList`, `LinkedList`, `Vector`, `Stack`
- For **Queue**: `PriorityQueue`, `Deque`, `ArrayDeque`
- For **Set**: `HashSet`, `LinkedHashSet`, `SortedSet`, `TreeSet`

4. Iterator Interface

The iterator interface is used to traverse the elements of a collection. It provides a uniform way to access elements of different collection types without exposing their internal implementation details.

4.1 Purpose of the Iterator Interface

1. **Traversal:** It allows sequential access to collection elements without needing to know the internal data structure.
2. **Uniformity:** It offers a common way to iterate over different collection types.
3. **Safe Removal:** It supports safe removal of elements during iteration.
4. **Enhanced For-Loop Support:** It is used implicitly in the enhanced `for` loop, making code more concise and readable.

5. Collection Interface

The `Collection` interface is the root interface in the Java Collections Framework. It represents a group of objects called elements and provides a common set of operations that can be performed on collections regardless of their actual implementation.

Important points:

- It serves as the fundamental building block of the framework.
- It does not specify any particular ordering of elements.
- It allows duplicate elements.
- It provides a unified way to work with groups of objects.

6. Difference Between Collection and Collections

Both **Collection** and **Collections** belong to the `java.util` package, but they are not the same thing.

Collection	Collections
<code>java.util.Collection</code> is an interface.	<code>java.util.Collections</code> is a class.
It is used to represent a group of objects as a single entity.	It is used to define various utility methods for collection objects.
It is the root interface of the Collection Framework.	It is a utility class.
It is used to derive the data structures of the Collection Framework.	It contains various static methods for data-structure manipulation.

7. List Interface

The **List** interface is a child interface of **Collection**. It is used for list-type data where ordered collections of objects are stored. A list allows duplicate elements.

It is implemented by classes such as **ArrayList**, **Vector**, and **Stack**. Since these classes implement **List**, a list object can be instantiated using any of these implementations.

7.1 Key Characteristics of List Interface

1. Ordered collection
2. Indexed access
3. Dynamic size
4. Iterable
5. Search operations

8. ArrayList

The **ArrayList** class is a resizable array found in the `java.util` package. Unlike a built-in array, its size can grow or shrink as elements are added or removed.

8.1 How ArrayList Works

An `ArrayList` internally uses a regular array. When a new element is added, it is placed into that internal array. If the current array is full, a larger array is created, elements are copied into it, and the old array is discarded.

Important features:

- Implements the `List` interface
- Uses a dynamic array internally
- Maintains insertion order
- Is not synchronized
- Supports random access
- Allows duplicate elements

8.2 Example: ArrayList Traversal with Iterator

```
import java.util.*;

class TestJavaCollection1 {
    public static void main(String args[]) {
        ArrayList<String> list = new ArrayList<String>();
        list.add("Ravi");
        list.add("Vijay");
        list.add("Ravi");
        list.add("Ajay");

        Iterator<String> itr = list.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}
```

Output:

```
Ravi
Vijay
Ravi
Ajay
```

9. LinkedList

The `LinkedList` class is a collection that can contain many objects of the same type, just like `ArrayList`. It has the same major methods because both implement list-related behavior, but the internal representation is different.

9.1 How LinkedList Works

A linked list stores data in containers (nodes). The list has a link to the first container, and each container has a link to the next one. To add a new element, a new container is created and linked into the structure.

The unit material describes `LinkedList` as using a doubly linked list internally.

Important features:

- Stores elements using linked nodes
- Can contain duplicate elements
- Maintains insertion order
- Is not synchronized
- Fast manipulation because shifting is not required

9.2 Example: LinkedList Traversal with Iterator

```
import java.util.*;

public class TestJavaCollection2 {
    public static void main(String args[]) {
        LinkedList<String> al = new LinkedList<String>();
        al.add("Ravi");
        al.add("Vijay");
        al.add("Ravi");
        al.add("Ajay");

        Iterator<String> itr = al.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}
```

Output:

```
Ravi
Vijay
Ravi
Ajay
```

10. Vector

The `Vector` class implements a growable array of objects. It is a legacy class, but it is compatible with collections and is found in the `java.util` package. It implements the `List` interface.

Important points:

- It implements a dynamic array.
- It can grow or shrink as required.
- It supports indexed access like an array.
- It is similar to `ArrayList`, but it is synchronized.
- It contains some legacy methods not part of the modern Collection Framework.
- It maintains insertion order.
- Because it is synchronized, it is rarely preferred in non-threaded environments due to poorer performance for add, search, delete, and update operations.
- Iterators returned by `Vector` are fail-fast and throw `ConcurrentModificationException` under concurrent modification.

10.1 Example: Vector

```
import java.util.*;

public class TestJavaCollection3 {
    public static void main(String args[]) {
        Vector<String> v = new Vector<String>();
        v.add("Ayush");
        v.add("Amit");
        v.add("Ashish");
        v.add("Garima");

        Iterator<String> itr = v.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}
```

Output:

```
Ayush
Amit
Ashish
Garima
```

11. Stack

The Java Collection Framework provides a `Stack` class that models the stack data structure. It is based on the Last-In-First-Out (LIFO) principle.

In addition to basic push and pop operations, the class supports methods such as `empty()`, `search()`, and `peek()`. The class extends `Vector`, so it contains all methods of `Vector` as well.

Important points:

- It implements the last-in-first-out structure.
- It behaves as a subclass of **Vector**.
- Common operations include `push()`, `pop()`, and `peek()`.

11.1 Example: Stack

```
import java.util.*;

public class TestJavaCollection4 {
    public static void main(String args[]) {
        Stack<String> stack = new Stack<String>();
        stack.push("Ayush");
        stack.push("Garvit");
        stack.push("Amit");
        stack.push("Ashish");

        stack.push("Garima");
        stack.pop();

        Iterator<String> itr = stack.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}
```

Output:

```
Ayush
Garvit
Amit
Ashish
```

12. Queue Interface

The **Queue** interface is present in the `java.util` package and extends the **Collection** interface. It is used to hold elements that are to be processed in FIFO (First-In-First-Out) order.

It is an ordered list of objects in which insertion generally takes place at one end and deletion takes place at the other.

Important points:

- It maintains first-in-first-out order.
- It is commonly implemented by **PriorityQueue**, **LinkedList**, **Deque**, and **ArrayDeque**.
- Neither **PriorityQueue** nor **LinkedList** is thread-safe.

- `PriorityBlockingQueue` can be used where thread-safe priority processing is needed.

12.1 Instantiation Examples

```
Queue<String> q1 = new PriorityQueue<String>();  
Queue<String> q2 = new ArrayDeque<String>();
```

13. PriorityQueue

A `PriorityQueue` is used when objects need to be processed according to priority rather than strict insertion order.

Although queue structures are usually associated with FIFO, a priority queue processes elements according to priority. It is based on a priority heap. The ordering may be determined either by:

- natural ordering of elements, or
- a `Comparator` supplied at queue construction time.

In the example shown, an element with a maximum ASCII value would have the highest priority according to the configured ordering.

Important points:

- Implements the `Queue` interface
- Holds elements to be processed by priority
- Does not allow `null` values

13.1 Example: PriorityQueue

```
import java.util.*;  
  
public class TestJavaCollection5 {  
    public static void main(String args[]) {  
        PriorityQueue<String> queue = new PriorityQueue<String>();  
        queue.add("Amit Sharma");  
        queue.add("Vijay Raj");  
        queue.add("JaiShankar");  
        queue.add("Raj");  
  
        System.out.println("head:" + queue.element());  
        System.out.println("head:" + queue.peek());  
        System.out.println("iterating the queue elements:");  
  
        Iterator<String> itr = queue.iterator();  
        while (itr.hasNext()) {  
            System.out.println(itr.next());  
        }  
    }  
}
```

```
        queue.remove();
        queue.poll();

        System.out.println("after removing two elements:");
        Iterator<String> itr2 = queue.iterator();
        while (itr2.hasNext()) {
            System.out.println(itr2.next());
        }
    }
}
```

Output:

```
head:Amit Sharma
head:Amit Sharma
iterating the queue elements:
Amit Sharma
Raj
JaiShankar
Vijay Raj
after removing two elements:
Raj
Vijay Raj
```

14. Deque Interface

The Deque (double-ended queue) interface is a subinterface of `Queue`. It provides a double-ended queue where elements can be added and removed from both ends.

Important features:

- Extends the `Queue` interface
- Allows insertion from both ends
- Allows deletion from both ends
- Can be used to implement different algorithms and data structures

14.1 Instantiation Example

```
Deque<String> d = new ArrayDeque<String>();
```

15. ArrayDeque

`ArrayDeque` is an implementation of the `Deque` interface that uses a resizable array to store elements.

It is described in the unit material as a more efficient alternative to the traditional stack class for double-ended operations.

Important points:

- Uses a resizable array internally
- Supports insertion and deletion from both ends
- Provides constant-time performance for many add/remove operations at both ends
- Is faster than `ArrayList` and `Stack` in the use case described
- Has no capacity restrictions

15.1 Example: ArrayDeque

```
import java.util.*;

public class TestJavaCollection6 {
    public static void main(String[] args) {
        Deque<String> deque = new ArrayDeque<String>();
        deque.add("Gautam");
        deque.add("Karan");
        deque.add("Ajay");

        for (String str : deque) {
            System.out.println(str);
        }
    }
}
```

Output:

```
Gautam
Karan
Ajay
```

16. Set Interface

The `Set` interface is present in the `java.util` package and extends the `Collection` interface. It represents an unordered collection of objects in which duplicate values cannot be stored.

The interface implements the mathematical notion of a set. It inherits methods from `Collection` and adds the restriction that duplicate elements are not allowed.

Important points:

- Duplicate elements are not allowed.
- At most one `null` value can be stored.
- Implementations include `HashSet`, `LinkedHashSet`, and `TreeSet`.
- Related interfaces include `SortedSet` and `NavigableSet`.

`NavigableSet` extends `SortedSet`, and `TreeSet` provides navigation through a self-balancing tree structure.

16.1 Instantiation Examples

```
Set<String> s1 = new HashSet<String>();
Set<String> s2 = new LinkedHashSet<String>();
Set<String> s3 = new TreeSet<String>();
```

17. HashSet

The `HashSet` class implements the `Set` interface and is backed by a hash table, which is internally based on a `HashMap` instance.

Important points:

- No guarantee is made about iteration order.
- The order is not necessarily constant over time.
- It permits one null element.
- Basic operations such as `add()`, `remove()`, `contains()`, and `size()` are efficient when the hash function distributes elements well.
- Duplicate values are not allowed.
- Elements are inserted based on hash code rather than insertion order.
- It implements `Cloneable` and `Serializable`.

17.1 Declaration of HashSet

```
public class HashSet<E> extends AbstractSet<E>
    implements Set<E>, Cloneable, Serializable
```

Here, `E` is the type of elements stored in the set.

17.2 Example: HashSet

```
import java.util.*;

public class TestJavaCollection7 {
    public static void main(String args[]) {
        HashSet<String> set = new HashSet<String>();
        set.add("Ravi");
        set.add("Vijay");
        set.add("Ravi");
        set.add("Ajay");

        Iterator<String> itr = set.iterator();
        while (itr.hasNext()) {
```

```
        System.out.println(itr.next());  
    }  
}  
}
```

Output:

```
Vijay  
Ravi  
Ajay
```

18. LinkedHashSet

LinkedHashSet is an ordered version of **HashSet**. It maintains a doubly linked list across all elements so that insertion order is preserved.

Important points:

- Extends **HashSet**
- Implements the **Set** interface
- Contains unique elements
- Maintains insertion order
- Permits **null** elements

When iterating through a **LinkedHashSet**, elements are returned in the order in which they were inserted.

19. SortedSet

The **SortedSet** interface extends **Set**. It adds the requirement that elements must be stored in sorted order.

This interface represents the mathematical set along with sorted-storage behavior.

20. TreeSet

TreeSet is one of the most important implementations of **SortedSet**. It uses a tree for storage. The ordering of elements is maintained either by natural ordering or by an explicit comparator.

The ordering must be consistent with **equals()** if the set is to correctly implement the contract of the **Set** interface.

Important features:

1. Sorted order
2. Unique elements
3. Efficient operations

4. Iterating in sorted order
5. Use of Red-Black tree

21. Map Interface

A map stores elements as key–value pairs, where each key is associated with a corresponding value.

Important property:

- **Uniqueness of keys:** Each key in a map is unique.

22. HashMap

The unit pages in the extracted text are incomplete for this subsection, but from the available material and flow of the unit, **HashMap** is treated as the standard map implementation used to store data in key–value form with fast insertion, search, and deletion.

Based on the surrounding discussion in the provided unit:

- It stores data in key–value pairs.
- Keys are unique.
- It is the reference point used to compare **LinkedHashMap** and **Hashtable**.
- It does not maintain insertion order.
- It is generally preferred over **Hashtable** when synchronization is not required.

23. LinkedHashMap

The **LinkedHashMap** class is similar to **HashMap** but provides the additional feature of maintaining the order of insertion.

HashMap offers quick insertion, searching, and deletion, but does not maintain insertion order, whereas **LinkedHashMap** preserves that order.

Important points:

- Implements **Map<K,V>**
- Extends **HashMap<K,V>**
- Maintains insertion order
- Stores data in the form of nodes
- Its implementation is described as similar to a doubly linked list

Here:

- K is the key object type
- V is the value object type

24. Hashtable

The `Hashtable` class implements a hash table that maps keys to values. Any non-`null` object can be used as a key or value. To store and retrieve objects successfully, the key objects must correctly implement `hashCode()` and `equals()`.

The material states that `Hashtable` was part of the original Java Collections world and was introduced in Java 1.0. It is now generally considered obsolete and its use is discouraged.

Reasons:

- It was designed before the modern Collections Framework.
- It is synchronized, which can lead to slower performance.
- Modern alternatives such as `HashMap` and `ConcurrentHashMap` are usually preferred.

Additional points:

- It is similar to `HashMap`, but synchronized.
- It stores key/value pairs in a hash table.
- The key is hashed, and the hash code determines the storage index.
- `HashMap` does not provide `Enumeration`, while `Hashtable` provides a not fail-fast `Enumeration`.

25. Sorting in Java

Although classical sorting algorithms such as selection sort, bubble sort, insertion sort, radix sort, and bucket sort exist, Java also provides convenient sorting through its data structures and utility methods.

Ways of sorting in Java listed in the unit:

1. Using loops
2. Using `sort()` method of `Arrays` class
3. Using `sort()` method of `Collections` class
4. Sorting on a subarray

26. Comparable Interface

The `Comparable` interface is used to define the natural ordering of objects of a user-defined class.

Important points:

- It is found in the `java.lang` package.
- It contains one method: `compareTo(Object)`.
- It provides a single sorting sequence only.

- Sorting can be done on one data member at a time, such as roll number, name, or age.

27. Sorting with Collections.sort()

The `Collections` class provides static methods for sorting collection elements.

Important points:

- If collection elements are of set or map type, `TreeSet` or `TreeMap` may be used for sorted storage.
- The `Collections` class provides sorting methods mainly for list-type elements.
- `String` and wrapper classes implement `Comparable` by default.

27.1 Method Signature

```
public void sort(List list)
```

This method is used to sort list elements, and the list elements must be of comparable type.

28. Comparator Interface

A `Comparator` is used to order objects of user-defined classes. A comparator object can compare two objects of the same class.

28.1 Syntax

```
public int compare(Object obj1, Object obj2)
```

Suppose an array or `ArrayList` contains user-defined objects with fields like roll number, name, address, date of birth, and so on. If sorting is required on different criteria such as roll number or name, a comparator is useful.

There are two approaches:

- **Method 1:** Write a custom sort function for each criterion. This is repetitive.
- **Method 2:** Use the `Comparator` interface. This avoids rewriting sorting logic for each field.

Additional note from the material:

- `Comparator` is in the `java.util` package.
- It provides methods `compare(Object obj1, Object obj2)` and `equals(Object element)`.
- It supports sorting by different data members such as roll number, name, or age.

29. Sorting List Elements Using Comparator

The method form for sorting list elements using a comparator:

```
public void sort(List list, ComparatorClass c)
```

To sort a given list, the comparator class must implement the `Comparator` interface.

29.1 How the sort() Method Works

Internally, the sort method uses the comparison logic to determine ordering. It asks, in effect, which of two elements is greater. The comparison method returns:

- -1 if the first element is less than the second
- 0 if both elements are equal
- 1 if the first element is greater than the second

The sorting routine uses this result to decide whether elements should be swapped.

30. Properties Class

A properties file is useful when data changes frequently, because the Java class does not have to be recompiled whenever the external property values change.

30.1 Advantages of a Properties File

- If data changes in the properties file, recompilation of the Java class is not required.
- It is useful for storing data that is expected to change frequently.

30.2 Important Note

Properties class does not inherit the concept of load factor from its superclass.

30.3 Declaration

```
public class Properties extends Hashtable<Object, Object>
```

30.4 Constructors of Properties

1. Properties()

Creates a properties object with no default values.

```
Properties p = new Properties();
```

2. Properties(Properties propDefault)

Creates a properties object that uses `propDefault` for default values.

```
Properties p = new Properties(propDefault);
```

31. Conclusion

This unit introduces the Java Collections Framework as a standardized architecture for storing, organizing, traversing, and manipulating groups of objects. It covers the core interfaces (`Collection`, `List`, `Queue`, `Set`, and `Map`), their common implementations, iterator-based traversal, and sorting support through `Comparable`, `Comparator`, and `Collections.sort()`.

The framework improves software development through reusable implementations, type safety, consistency, scalability, and efficient data handling.